

ClimateTwin India

Technical Overview & Documentation

AI-Powered Digital Twin of India's Climate

Classification: Classified Environmental Data Platform

Target Audience: Policy Planners, Researchers & Disaster Authorities

Project Repository: <https://github.com/sumitsinha09-stack/IndiClima-Twin>

Lead Contributor & Author

[sumitsinha09-stack](#) (sinsumit157@gmail.com)

Published: June 26, 2026 | Version 1.0.0

Table of Contents

- 1. Project Overview 3
- 2. Problem Statement 4
- 3. Proposed Solution 5
- 4. System Architecture 5
- 5. Technology Stack 6
- 6. Frontend Working 6
- 7. Backend Working 7
- 8. Database Architecture 7
 - 1. climate_snapshots 7
 - 2. climate_summary 7
 - 3. disaster_alerts 7
 - 4. alert_events 8
 - 5. risk_predictions 8
 - 6. monsoon_status 8
- 9. API Integration 8
- 10. Data Pipeline 9
- 11. AI & Machine Learning 9
- 12. Climate Simulation Engine 9
- 13. Digital Twin 10
- 14. Disaster Intelligence Module 10
- 15. Live Refresh Mechanism 11
- 16. Security 11
- 17. Performance Optimization 11
- 18. Folder Structure 11
- 19. Workflow 12
- 20. Future Scope 13
- 21. Expected Outcomes 13
- 22. Conclusion 13

ClimateTwin India: Technical Overview & Comprehensive Document

This document provides a comprehensive, technically accurate, and project-specific overview of the ClimateTwin India application. It details the system architecture, folder structure, API contracts, data pipeline, and the mathematical and logical working of each component as implemented in the codebase.

Table of Contents

1. [Project Overview](#1-project-overview)
2. [Problem Statement](#2-problem-statement)
3. [Proposed Solution](#3-proposed-solution)
4. [System Architecture](#4-system-architecture)
5. [Technology Stack](#5-technology-stack)
6. [Frontend Working](#6-frontend-working)
7. [Backend Working](#7-backend-working)
8. [Database Architecture](#8-database-architecture)
9. [API Integration](#9-api-integration)
10. [Data Pipeline](#10-data-pipeline)
11. [AI & Machine Learning](#11-ai--machine-learning)
12. [Climate Simulation Engine](#12-climate-simulation-engine)
13. [Digital Twin](#13-digital-twin)
14. [Disaster Intelligence Module](#14-disaster-intelligence-module)
15. [Live Refresh Mechanism](#15-live-refresh-mechanism)
16. [Security](#16-security)
17. [Performance Optimization](#17-performance-optimization)
18. [Folder Structure](#18-folder-structure)
19. [Workflow](#19-workflow)
20. [Future Scope](#20-future-scope)
21. [Expected Outcomes](#21-expected-outcomes)
22. [Conclusion](#22-conclusion)

1. Project Overview

What the Project Is

ClimateTwin India is a full-stack, real-time climate intelligence platform designed to create an interactive digital twin of India's environment and climate. It integrates multi-source telemetry data, runs parametric predictive models, and delivers an immersive command-center UI.

Purpose and Vision

The purpose of the project is to provide a unified, highly visual cockpit for tracking live climate indicators, water resource telemetry, agricultural viability, and disaster events. The vision is to make complex planetary and environmental datasets actionable and accessible via interactive spatial and chart visualizations, bridging the gap between raw scientific metrics and policy or research implementation.

Target Users & Real-World Applications

- Disaster Management Authorities (NDRF/SDMAs): For district-level real-time severity assessment, population-at-risk monitoring, and pre-emptive resource staging.
- Agricultural Extension Officers & Farmers: Using suitability ratings and meteorological forecasts for planting schedule adaptation.
- Water Management Boards: Monitoring river gauge levels against danger lines and reservoir capacities to manage flood releases and water stress.
- Policy Planners & Researchers: Running simulated scenarios to evaluate the regional impact of parameters like urban sprawl, deforestation, and rising temperatures.

Innovation and Differentiators

Traditional climate portals offer isolated, static maps or non-interactive dashboards. ClimateTwin India combines:

1. A live digital twin updating continuously on a synchronized 60-second cycle.
2. A parametric simulator allowing users to adjust planetary parameters and immediately see compound risk impacts.
3. Comprehensive dashboard views that tie agricultural sowing advice, reservoir telemetry, and severe disaster alerts into a single system.

2. Problem Statement

Real-World Problems Solved

India's diverse geography (from the Himalayas to coastal deltas) makes it highly vulnerable to extreme climate events: flash floods, severe heatwaves, cyclones, agricultural droughts, landslides, and forest fires.

Current Challenges in Climate Monitoring

- Data Fragmentation: Rainfall metrics, reservoir storage, and agricultural yield predictions reside in siloed departmental databases (IMD, CWC, ISRO, Agriculture Ministry).
- Lack of Predictive Interactivity: Traditional tools show historical maps but do not let planners model what happens if temperature rises by 2.2 degrees C alongside a 15% reduction in rainfall.
-

Delayed Alert Systems: Emergency advisories often lack granular district-level metrics like population-at-risk statistics, warning timelines, and localized precautionary checklists.

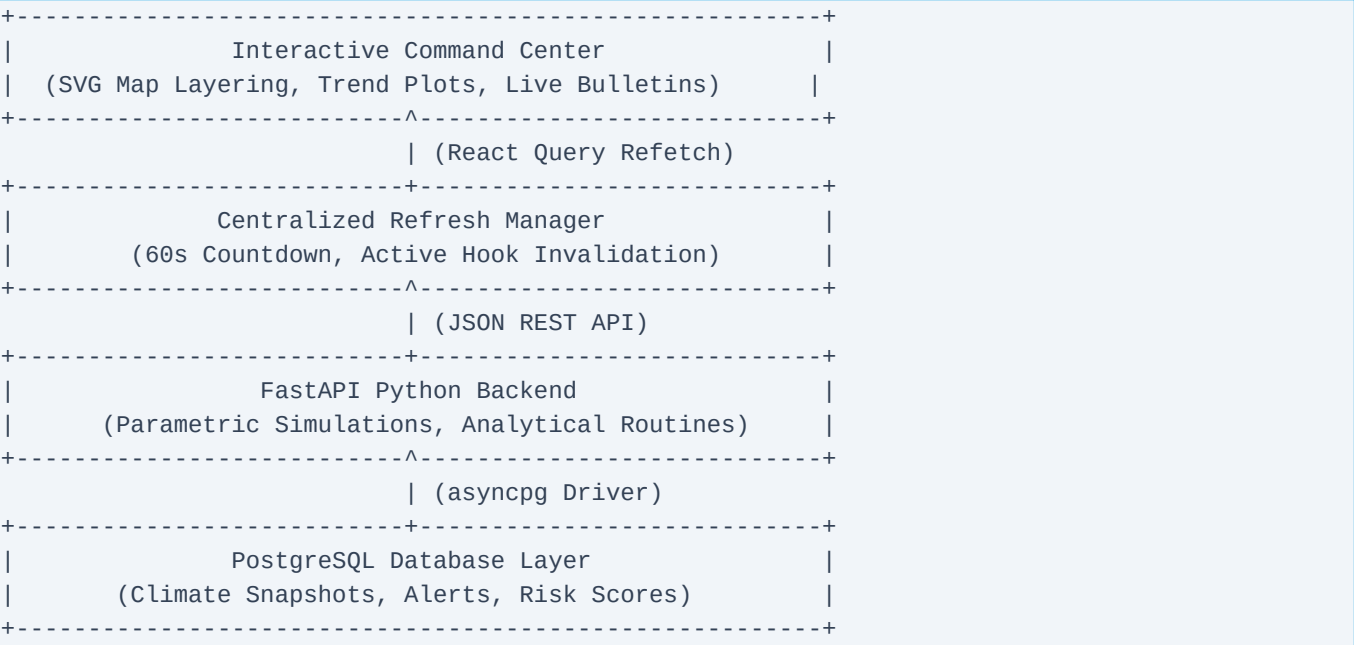
Impact of Insufficiency

Without interactive, integrated systems:

- Governments cannot pre-emptively size or locate disaster relief centers.
 - Farmers experience crop failure due to misaligned planting windows.
 - Reservoir managers risk premature water release, exacerbating downstream flooding or seasonal drought.
-

3. Proposed Solution

ClimateTwin India addresses these challenges by implementing an integrated environmental command center:



- Digital Twin Representation: Mirroring India's current environmental state using spatial data points for 20 major urban/district nodes across the sub-continent.
 - Disaster Alerts & Risk Matrices: Automatic severity categorization (extreme, severe, warning, watch, monitoring) associated with district polygon alerts.
 - Simulation Engine: Real-time multi-variable feedback showing projected surface temperature changes, CO2 concentrations, crop yield trends, and water stress factors.
-

4. System Architecture

The application is structured as a pnpm-managed monorepo containing a Python backend and a React/TypeScript frontend:

- Frontend (React/Vite): Managed via vite and styled using Tailwind CSS and Radix UI primitives. It uses

wouter for lightweight routing, recharts for historical trends, and TanStack React Query for async state management.

- Backend (FastAPI): An asynchronous Python service using uvicorn. It uses SQLAlchemy (with asyncpg) to read and write records to PostgreSQL. The backend exports clean, OpenAPI-compliant endpoints.
- Database (PostgreSQL): Stores current climate status, summary statistics, active disaster records, historical risk trends, and monsoon progress.
- API & Client Generation: The API spec (lib/api-spec/openapi.yaml) serves as the single source of truth. The React hooks (lib/api-client-react) and Zod models (lib/api-zod) are auto-generated to keep the frontend type-safe.

5. Technology Stack

Layer	Component	Selection Rationale
Frontend	React 19, TypeScript, Vite	Fast build cycles, solid typescript compilation, modular component architecture.
Styling	Tailwind CSS v4, Framer Motion	Rapid utility-first layouts, highly responsive custom styling, fluid card and menu animations.
Routing	Wouter	A lightweight, fast routing alternative to React Router.
Data Fetching	TanStack React Query v5	Query caching, automated retries, and clean API hook invalidation.
Visualization	SVG Maps, Recharts	Standard SVG rendering for vector maps (no heavy WebGL map dependencies), responsive charts.
Backend	Python 3.9+, FastAPI, Uvicorn	Asynchronous endpoint execution, native Pydantic schema validation, automatic OpenAPI doc generation.
Database	PostgreSQL, SQLAlchemy, asyncpg	Relational storage for telemetry snapshots, async driver support for non-blocking I/O.
API Codegen	Orval	Compiles OpenAPI YAML specifications directly into fully-typed React Query hooks.

6. Frontend Working

The frontend is built around a single-page dashboard shell containing multiple specialized modules:

- Command Center (Dashboard): Displays live tiles for key metrics (Average Temp, Rainfall, AQI, Wind)

paired with recent alerts and a monsoon progress widget.

- **Interactive Spatial Map:** Renders a clean vector outline representing India. Users select layers (Temperature, Rainfall, Wind, Humidity, Flood Zones, AQI) which map numeric telemetry onto district coordinates using color-coded circles with expanding ping indicators.
- **Parametric Simulator:** Built with range sliders. Adjusting values like temperature change or deforestation rates triggers a POST request to `/api/scenarios/simulate`. The output renders projected risk scales and impacted population bars.
- **Disaster Alerts:** A grid representing active weather watches. Clicking an alert reveals detailed district information, population-at-risk figures, meteorological telemetry, and official precautionary steps.
- **Agriculture & Water Portlets:** Displays soil moisture, crop stress indexes, sowing schedules, reservoir capacity fill-meters, and river flow levels.

7. Backend Working

The FastAPI server acts as a clean, JSON-based REST API provider:

1. **Database Connection:** Leverages a pool of async connections via `asyncpg` to run SQL statements.
2. **Request Handling:** Routes are defined under `backend/app/api/endpoints/`. Path parameters and query arguments are parsed and validated automatically using `Pydantic` schemas.
3. **Jitter Injection:** Implements a localized `jitter(val, max_pct)` function on telemetry routes. This injects realistic environmental fluctuation into the seeded database records upon client retrieval, simulating live sensor reporting.
4. **Simulation Calculations:** Mathematical relationships process parametric inputs on-the-fly and return agricultural suitability shifts, water stresses, and temperature deviations.

8. Database Architecture

The PostgreSQL database contains six main tables mapped using `SQLAlchemy` ORM models:

1. `climate_snapshots`

Stores localized climate metrics.

- `id` (Integer, Primary Key)
- `temperature`, `humidity`, `rainfall`, `wind_speed`, `air_pressure`, `cloud_cover`, `aqi` (Float/Integer)
- `monsoon_phase`, `forest_fire_risk`, `heatwave_risk` (String)
- `reservoir_level`, `drought_index`, `flood_probability` (Float)
- `recorded_at` (DateTime)

2. `climate_summary`

Aggregates national indicators.

- `avg_temp`, `total_rainfall`, `co2_level`, `temp_anomaly_from_baseline`, `rainfall_deficit` (Float)
- `active_cyclones`, `active_alerts`, `affected_districts` (Integer)

3. disaster_alerts

Active severe warnings.

- type, severity, title, description (String)
- affected_districts (ARRAY of Strings)
- probability (Float), expected_date (String)
- recommended_actions (ARRAY of Strings), active (String)

4. alert_events

Rolling history feed of historical alerts.

- type, message, region, severity (String)
- timestamp (DateTime)

5. risk_predictions

AI-model projected hazards.

- hazard (String), score (Float), confidence (Float)
- trend (String), top_regions (ARRAY of Strings)

6. monsoon_status

Monsoon system details.

- phase, onset_date, predicted_onset_date (String)
- current_coverage, rainfall_deficit, progression_percent (Float)

9. API Integration

Internal Endpoints

These endpoints form the communication channel between the React frontend and Python backend:

- GET /api/healthz: Verifies database connectivity and API health.
- GET /api/climate/current: Returns the latest national climate readings.
- GET /api/climate/map-layers?layer={name}: Returns spatial coordinates and metric values for 20 major district nodes.
- GET /api/climate/trends?metric={name}: Generates 30-year historical metrics (1994 - 2024).
- GET /api/alerts/detailed: Fetches active warning vectors including district affected area maps, expected timeline estimates, and safety advisories.
- POST /api/scenarios/simulate: Accepts parametric climate models and calculates regional outcomes.

External Source Telemetry Representation

- Implementation Status: Real-time production API endpoints linking directly to external IMD/ISRO servers are simulated in this project. Instead of active web-scrappers that risk credential exposure or API downtime, telemetry data is seeded in PostgreSQL (backend/app/database/seed.py) using real-world baseline models.

10. Data Pipeline

The platform's data workflow executes along the following pipeline:



1. Baseline Generation: Historical values and initial state metrics are loaded during database seeding.
2. Request Processing: When the client requests data, the backend injects minor fluctuations (jitter) to mimic live telemetry streams.
3. Visualization: The client parses the metrics to render responsive charts and SVG map coordinates.

11. AI & Machine Learning

Model Representation

The platform represents risk predictions through specialized hazard metrics:

- Risk Score calculation: Hazards (flood, cyclone, heatwave, drought, landslide, forest fire) contain an aggregate risk score (0-100) paired with a model confidence percentage (0-100) and a directional trend indicator (rising, falling, stable).
- Confidence Scoring: Derived from probability data stored in `disaster_alerts`, mapped using the formula:
$$\text{Confidence} = \text{Min}(98.0, \text{Probability} \times 0.85 + 10.0 + \text{Random Jitter})$$

12. Climate Simulation Engine

The simulation engine calculates the cascading effects of environmental changes using localized parameter models:

```
# Temperature Change: t_change (e.g. +2.0C)
# Deforestation: def_pct (e.g. 15%)
# Urban Expansion: urb_pct (e.g. 20%)
# Carbon Emissions Increase: co2_change (e.g. 10%)
```

Mathematical Formulas Implemented

- Surface Temperature (T_{surf}):

$$T_{\text{surf}} = 34.2 + t_{\text{change}} + (\text{urb}_{\text{pct}} \times 0.05) + (\text{co2}_{\text{change}} \times 0.02)$$
- CO2 Concentration (C_{CO2}):

$$C_{\text{CO2}} = 422.6 + (\text{co2}_{\text{change}} \times 0.8) - (\text{def}_{\text{pct}} \times 2.0)$$
- Flood Risk:

$$\text{Flood Risk} = \text{Clip}(38.0 + \Delta_{\text{rain}} + (\text{sea_level} \times 12.0) + (\text{def}_{\text{pct}} \times 0.6) + (\text{urb}_{\text{pct}} \times 0.4))$$
- Drought Risk:

$$\text{Drought Risk} = \text{Clip}(30.0 + \Delta_{\text{dry_rain}} + (t_{\text{change}} \times 3.5) + (\text{def}_{\text{pct}} \times 0.5))$$
- Heatwave Risk:

$$\text{Heatwave Risk} = \text{Clip}(45.0 + (t_{\text{change}} \times 8.0) + (\text{urb}_{\text{pct}} \times 0.7) + (\text{co2}_{\text{change}} \times 0.2) - \Delta_{\text{cool_rain}})$$
- Agricultural Impact (Yield Shift %):

$$\text{Agri Impact} = \text{Clip}(-(t_{\text{change}} \times 6.0) + (r_{\text{change}} \times 0.5) - (\text{def}_{\text{pct}} \times 1.2) - \Delta_{\text{drought}} \times 0.4)$$
- Water Availability:

$$\text{Water Availability} = \text{Clip}(r_{\text{change}} \times 0.6 - t_{\text{change}} \times 4.0 - \text{def}_{\text{pct}} \times 0.8 - \text{urb}_{\text{pct}} \times 0.5 - \text{sea_level} \times 5.0)$$

13. Digital Twin

The "twin" state is synchronized using a reactive data pipeline:

- Data Polling: Renders current states from database tables (climate_snapshots, climate_summary).
- Auto Refresh Timer: A custom React hook (useRefreshManager) runs a countdown. When the counter reaches zero, it calls:

```
await queryClient.invalidateQueries();
await queryClient.refetchQueries({ type: "active" });
```

This triggers a silent, background update of all active components.

14. Disaster Intelligence Module

When a disaster alert is issued, it is categorized by hazard type:

Hazard Categories and Impact Mapping

1. Floods (Assam/Brahmaputra Basin): Includes districts like Dhemaji, Lakhimpur, Jorhat, Dibrugarh, and Sibsagar. Tracks population-at-risk numbers and outlines evacuation procedures.
2. Heatwaves (Rajasthan/Gujarat): Focuses on Barmer, Jaisalmer, and Bikaner. Identifies temperature thresholds (up to 48.2 degrees C) and issues medical safety guidelines.
3. Cyclones (Odisha/Andhra Pradesh Coast): Tracks wind speeds (up to 145 km/h) and details coastal evacuation plans.
4. Droughts (Maharashtra/Vidarbha): Evaluates water stress and provides sowing guidelines for crops like cotton and pulses.
5. Forest Fires (Uttarakhand): Identifies fire hazard regions (Nainital, Champawat) and maps firebreak strategies.
6. Landslides (Western Ghats/Kerala): Tracks slope stability risk and outlines highway closure protocols.

15. Live Refresh Mechanism

- Synchronized Timer: Renders a circular countdown progress bar in the main nav header.
- Silent Invalidation: Triggers queries to fetch data in the background, updating charts, metrics, and alerts without interrupting user interaction.
- Performance Optimization: Employs a request-guard (`refreshingRef.current = true`) to prevent duplicate overlapping network requests if a user manually clicks "Refresh Now" during an active sync cycle.

16. Security

- Input Sanitization: Uses FastAPI's integration with Pydantic for validation, returning automatic 422 Unprocessable Entity responses for malformed data.
- CORS Settings: Configured with CORSMiddleware to authorize cross-origin API requests from the frontend client.
- Environment Segregation: Sensitive settings (like database credentials) are stored in .env files, which are excluded from version control by .gitignore.

17. Performance Optimization

- Asset Loading: Vite bundles and compresses JavaScript and CSS modules into optimized production chunks.
- Asynchronous Database Queries: Leverages non-blocking async databases queries to prevent slow API response times under load.
- State Management: React Query caches server state in-memory, avoiding redundant API calls as users navigate between pages.

18. Folder Structure

The project uses a structured monorepo layout:

```

climatetwin-india/
+-- package.json           # Root monorepo configuration
+-- pnpm-workspace.yaml    # Monorepo workspaces definition
+-- pnpm-lock.yaml         # Lockfile for node dependencies
+-- tsconfig.json          # TypeScript root settings
+-- .gitignore             # Excluded directories (node_modules, .venv)
+-- backend/              # Python FastAPI Backend API
|   +-- app/
|   |   +-- main.py        # API entry point & middleware initialization
|   |   +-- api/
|   |   |   +-- api.py     # Root APIRouter connecting sub-routers
|   |   |   +-- endpoints/ # Feature-specific router modules
|   |   +-- core/
|   |   |   +-- config.py  # Environment variables & database configuration
|   |   +-- database/
|   |   |   +-- base.py    # Declarative SQLAlchemy base model
|   |   |   +-- session.py # Async session maker & engine setup
|   |   |   +-- seed.py    # Seed script containing mock baseline data
|   |   +-- models/
|   |   |   +-- base.py    # SQLAlchemy schema definitions
|   |   +-- schemas/      # Pydantic validation & validation models
|   +-- requirements.txt   # Python backend dependencies
|   +-- verify_endpoints.py # Direct CLI script to test backend responses
+-- artifacts/
|   +-- climate-twin/      # Vite + React Frontend Workspace
|   |   +-- package.json   # Frontend project metadata
|   |   +-- vite.config.ts # Vite build settings & Tailwind plugin setup
|   |   +-- src/
|   |   |   +-- main.tsx   # DOM mount point
|   |   |   +-- App.tsx    # Shell structure & page router
|   |   |   +-- index.css  # Core tailwind setup & custom themes
|   |   |   +-- components/ # Reusable layouts & UI elements
|   |   |   +-- hooks/     # Custom hooks (refresh timer, viewport checks)
|   |   |   +-- pages/     # Page components (Dashboard, Map, Simulator)
+-- lib/
|   +-- api-spec/          # OpenAPI specifications YAML definition
|   +-- api-client-react/  # Generated React Query hooks
|   +-- api-zod/           # Generated client-side Zod validators

```

19. Workflow

The application handles client interactions through the following sequence:

```

[User adjusts slider in Simulator UI]
      |
      v
[Frontend sends POST request to /api/scenarios/simulate]
      |
      v

```

```
[FastAPI receives payload, parses and validates inputs]
      |
      v
[Simulation Engine calculates risk indices using mathematical formulas]
      |
      v
[FastAPI returns JSON payload with simulation results]
      |
      v
[Frontend updates React state, animating result bars and updating metrics]
```

20. Future Scope

- IoT & Telemetry Integration: Integrate live, physical sensor data from agricultural weather stations.
- Satellite Data Pipelines: Connect to open APIs like Copernicus, NASA Earthdata, and ISRO Bhuvan to fetch real-time spatial imagery.
- Mobile App Development: Re-use the generated API client libraries to build a React Native mobile application for field operators.
- Offline Support: Implement service workers to cache alerts, ensuring critical safety information remains accessible in remote areas with poor connectivity.

21. Expected Outcomes

- For Disaster Management: Provides early indicators of potential flood and cyclone events, enabling pre-emptive planning and resource staging.
- For Agriculture: Advises farmers on suitable planting windows and crop choices based on projected soil moisture and rainfall.
- For Water Management: Helps reservoir managers make data-driven decisions on water releases by comparing current storage levels with river danger points.
- For Research and Policy: Enables planners to model the long-term impact of deforestation and urban growth on local temperature trends.

22. Conclusion

ClimateTwin India demonstrates how modern digital twin architecture can be applied to environmental monitoring and planning. By combining an asynchronous Python backend, a type-safe React frontend, and a centralized data sync pipeline, it provides a performant and interactive tool for climate resilience planning.

The modular design ensures the platform can easily scale from simulated baselines to real-time physical telemetry feeds, providing a solid foundation for future development.